

$$a = 2^0$$

data type

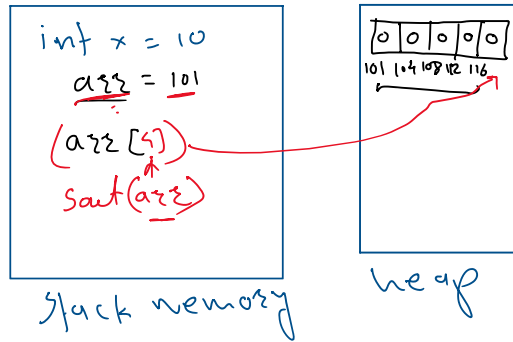
```
graph TD; Root[ ] --> Primitive[primitive]; Root --> NonPrimitive[non primitive]; Primitive --> Integer[integer]; Primitive --> Float[float]; Primitive --> Char[char]; Primitive --> Short[short]; Primitive --> Byte[byte]; Primitive --> Long[long]; Primitive --> Double[double]; Primitive --> Bool[bool]; NonPrimitive --> Array[Array]; NonPrimitive --> String[string]; NonPrimitive --> Example["int ar [ ]"];
```

↓
primitive
↳ integer
float
char
short
byte
long
double
bool

↓
non primitive
↳ Array
string
int ar []

int x = 10
int y = 101

int arr [] = new int[5]


$$\begin{array}{r} 127 \\ 1010 \end{array} \quad \begin{array}{r} 128 \\ \underline{11101} \end{array}$$

A hand-drawn diagram of a 4-bit register. It consists of four adjacent rectangular cells. The first cell contains the number '1', the second cell contains '1', the third cell contains '0', and the fourth cell contains '1'.

```
byte x = 300  
short(x)  
char ch = 'f'
```

() (())

Syntax
 $\hookrightarrow$ datatype variableName = value;
 ① ② ③ $\nwarrow$ initialization

variable declaration

```
byte a = 100;
short b = 200;
int x = 28100200;
long l = 100000000L;
```

```
short b = 200,
int x = 28100200
long l = 100000000L;
```

```
int a = 20
int b = 10
int sum = a + b
cout(sum) -> 30
```

Special powers of / & % by powers of 10

22++

```
int a = 20
cout(a++) -> 20
cout(a) -> 21
```

post-increment

↳ first use the value and then increase

variable++

```
int a = 20
cout(a) -> 20
```

pre-increment

↳ first increase the value and then use it

++variable

```
int a = 20
cout(++a) -> 21
```

Relational operators

↳ (<, >, ==, !=) -> boolean
↳ True/False

```
cout(20 < 30) -> True
```

```
cout(20 < 20) -> False
```

```
cout(20 < 20) -> False
```

~
cout (20 < 20) → false
cout (20 != 20) → false
cout (25 == 25) → true